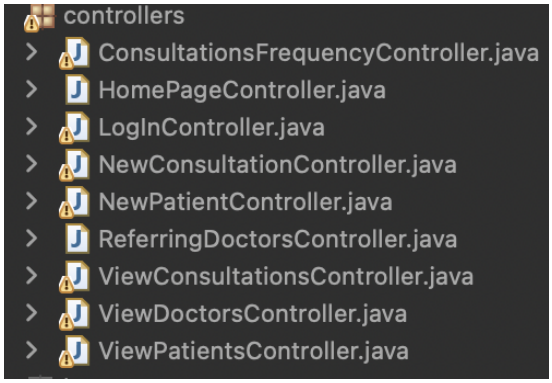
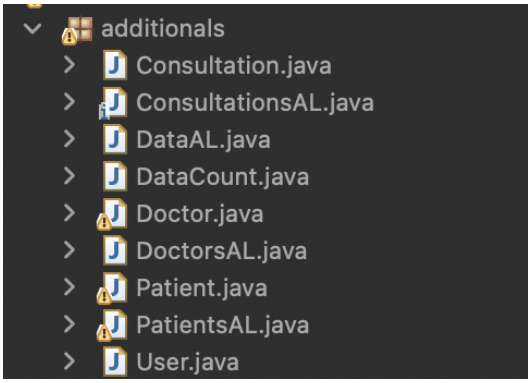
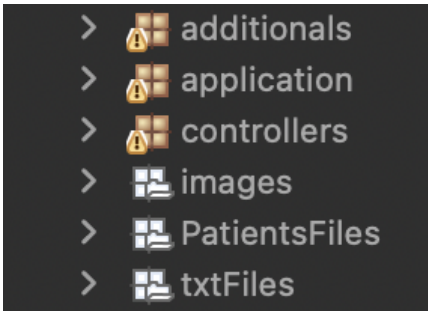
Criterion C

File Divisions







This project was divided into several files to ensure a clear project to work on and a smooth development of the code. Having this way of organizing archives also ensured that coming back to the code after some time becomes an easy task, increasing efficiency in the coding process.

OOP

Classes

Creating classes for every person involved and consultations was a way to organize all crucial information for each person and consultations, as well as associating information with each individual correctly and efficiently. Each class had as an instance variable important information needed to create an object of that person or consultation, which also helped with the establishment of which data is important to associate with which person or consultation.

Object creation.

```
public class Main extends Application {

    //Stage
    public static Stage mainstage;

    //Scenes
    public static Scene logIn;
    public static Scene homePage;
    public static Scene newPatient;
    public static Scene viewPatients;
    public static Scene newConsultations;
    public static Scene viewDoctors;
    public static Scene viewConsultations;
    public static Scene referringDoctors;
    public static Scene consultationsFrequency;

    //Controllers
    public static controllers.HomePageController hp;
    public static controllers.NewPatientController nP;
    public static controllers.ViewPatientsController vP;
    public static controllers.NewConsultationController nC;
    public static controllers.ViewDoctorsController vD;
    public static controllers.ViewConsultationsController vC;
    public static controllers.ReferringDoctorsController rD;
    public static controllers.ConsultationsFrequencyController cF;


    //Object creation;
    public static User currUser = new User ("1", "Dr. Manuel Bazaldua", "1");
    public static DoctorsAL doctorsAl = new DoctorsAL();
    public static PatientsAL patientsAl = new PatientsAL();
    public static ConsultationsAL consultationsAl = new ConsultationsAL();
    public static ConsultationsAL consultationsHistoryAl = new ConsultationsAL();
    public static DataAL doccountAl = new DataAL();
    public static DataAL consultationsCountAl = new DataAL();

    public static String monthsOfTheYear [] = {"None", "January", "February","March","April", "May","June", "July","August", "September", "October","November", "December"};
```

Crucial objects were created in the Main class. This is because these are essential objects that were going to be used in a lot of different controller classes. Especially with the creation of the Array Lists, creating them in the Main class, centralizing important information, and

ensuring memorability when developing the code. Also, correcting mistakes became an easy task.

```
public static void updatePatientsAlCsvFile() {
    try {
        PrintWriter writer = new PrintWriter (new FileWriter("src/txtFiles/PatientsAlFile.csv"));

        for (int i = 0; i < application.Main.patientsAl.getPatientAl().size(); i ++) {
            writer.println(application.Main.patientsAl.getPatientAl().get(i));
        }

        writer.close();

    }catch( Exception e) {
        e.printStackTrace();
    }

}
```

```
public static void readPatientsAlCsvFile() {
    try {
        BufferedReader reader = new BufferedReader (new FileReader("src/txtFiles/PatientsAlFile.csv"));
        String line = reader.readLine();

        while (line != null) {
            //Posible addition of intance variables consultationDate and consultationTime

            String [] values = line.split("\t");
            DateTimeFormatter formatter = DateTimeFormatter.ofPattern("yyyy-MM-dd");
            LocalDate date = LocalDate.parse(values[2], formatter);
            Doctor temp = new Doctor(values[7]);
            int height = Integer.parseInt(values[11]);
            double weight = Double.parseDouble(values[12]);

            Patient tempPatient = new Patient(values[1],date, values[4],values[5],values[6],temp.getName(),
                    values[8],values[9],values[10],height,weight,values[14],values[15],values[18]);

            tempPatient.setCDiagnosis(values[16]);
            tempPatient.setcPrescription(values[17]);


            application.Main.patientsAl.addPatient(tempPatient);

            line = reader.readLine();
        }
        reader.close();
    }catch (Exception e) {
        e.printStackTrace();
    }

}
```

Methods for reading and updating information in their respective CSV files were created for consultations, consultations log, patients, and doctors. These methods were also created in the main class since these were methods that were going to be used throughout almost every controller class, and it was important to have everything in the same place.

Methods

Data Validation and General Functionality

```
void btnAddNewPatientClick(ActionEvent event) {
    //Required information filter
    if (txtName.getText() != null &&
            datePickDateOfBirth.getValue() != null &&
            cBoSex.getValue() != null &&
            cboMaritalStatus.getValue() != null &&
            cboInsuranceCompany.getValue() != null &&
            txtEmail.getText() != null &&
            txtCellPhone.getText() != null &&
            txtResidence.getText() != null &&
            cboReferredDoc.getValue() != null &&
            !txtHeight.getText().equals(null) &&
            !txtWeight.getText().equals(null) &&
            !txtAlergies.getText().equals(null) &&
            !txtPastMedicalProc.getText().equals(null)) {
        try { //Data type filter/Checker

            int tempHeight = Integer.parseInt(txtHeight.getText());
            double tempWeight = Double.parseDouble(txtWeight.getText());

            Patient tempPatient = new Patient(txtName.getText(),
                    datePickDateOfBirth.getValue(),
                    cboMaritalStatus.getValue(),
                    cBoSex.getValue(),
                    cboInsuranceCompany.getValue(),
                    cboReferredDoc.getValue().getName(),
                    txtEmail.getText(),txtCellPhone.getText(),
                    txtResidence.getText(),
                    tempHeight,
                    tempWeight,
                    txtAlergies.getText(),
                    txtPastMedicalProc.getText(),
                    txtOccupation.getText());


            //Addition of optional information.
            if (txtDiagnosis.getText() != null) {
                tempPatient.setCDiagnosis(txtDiagnosis.getText());
            }

            if (txtPrescription.getText() != null) {
                tempPatient.setcPrescription(txtPrescription.getText());
            }

            //Adding new patient to ArrayLists
            application.Main.patientsAl.addPatient(tempPatient);

            //Updating Tables
            application.Main.vP.updateNewPatientTable(application.Main.patientsAl.getPatientAl());
            application.Main.nC.updateNewConsultationTable(application.Main.patientsAl.getPatientAl());


             //File Writer.
             application.Main.updatePatientsAlCsvFile();

             Alert patientCreated = new Alert (AlertType.INFORMATION);
             patientCreated.setHeaderText(null);
             patientCreated.setTitle("Patient Created!");
             patientCreated.setContentText("New Patient Succesfully Created!");
             patientCreated.show();

             clearTemplate();



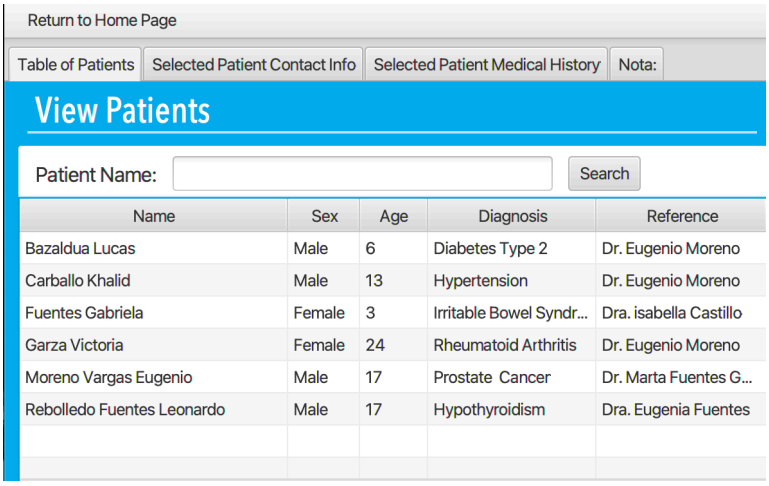
        }catch (Exception e) { //Detects wrong datatype input in both Height and Weight
            Alert notNum = new Alert (AlertType.ERROR);
            notNum.setHeaderText(null);
            notNum.setTitle("ERROR!");
            notNum.setContentText("Please input only numbers in 'Height' and 'Weight' section. ");
            notNum.show();
        }

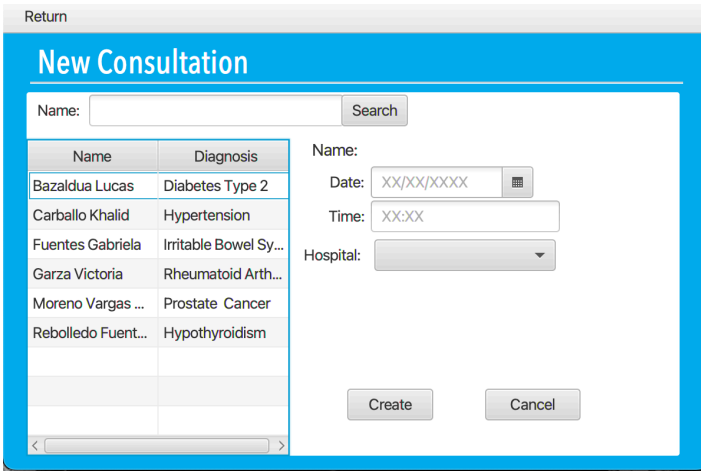
    }else { // Required information filter
        Alert notRequieredInfo = new Alert(AlertType.ERROR);
        notRequieredInfo.setHeaderText(null);
        notRequieredInfo.setTitle("ERROR!");
        notRequieredInfo.setContentText("Please fill all the required information");
        notRequieredInfo.show();
    }
}
```

Validation when creating a new Patient is essential to ensure that no essential piece of information is missing. All mandatory information was specifically requested by my client during the interview and consultations. I use a “try and catch” to also verify that the instances

“Height” and “Weight” are filled up correctly. This prevents the code from crashing, and also, ensures the information is in the correct format in order to use this data on the BMI formula, calculate it, and display it to the user. Alerts are also important, especially in this long scene, because they inform the user about their specific mistakes and it makes it easier for them to correct them and continue using the application normally.

My data validation does not check in the patient's Array List if there is already another patient with the same name. I made this decision because I considered that there was a great possibility that my client would tumble upon two patients with the exact same name.





Considering a potential issue, in the View Patients scene as well as in the New Consultation scene the user is able to see besides the Patient’s name, other information about the patient that can help the user differentiate one patient from another

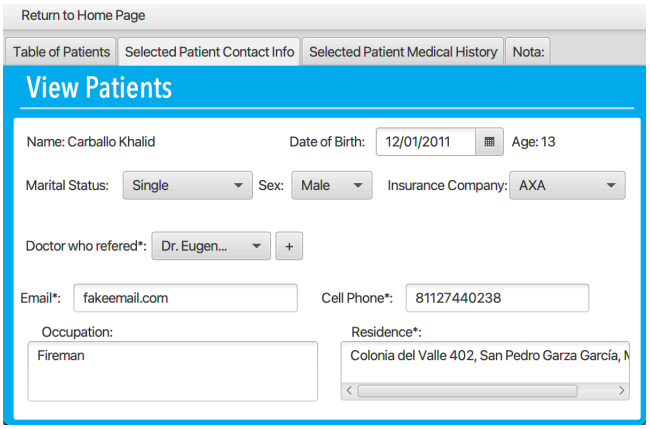
```
void btnEditOnClick(ActionEvent event) {
    int position = tablePatients.getSelectionModel().getSelectedIndex();

    if(position != -1) {
        Patient x = getPatientData();
        if (application.Main.patientsAl.binarySearchByName(x.getName()) != -1 ) {
            application.Main.patientsAl.getPatientAl().set(position, x);

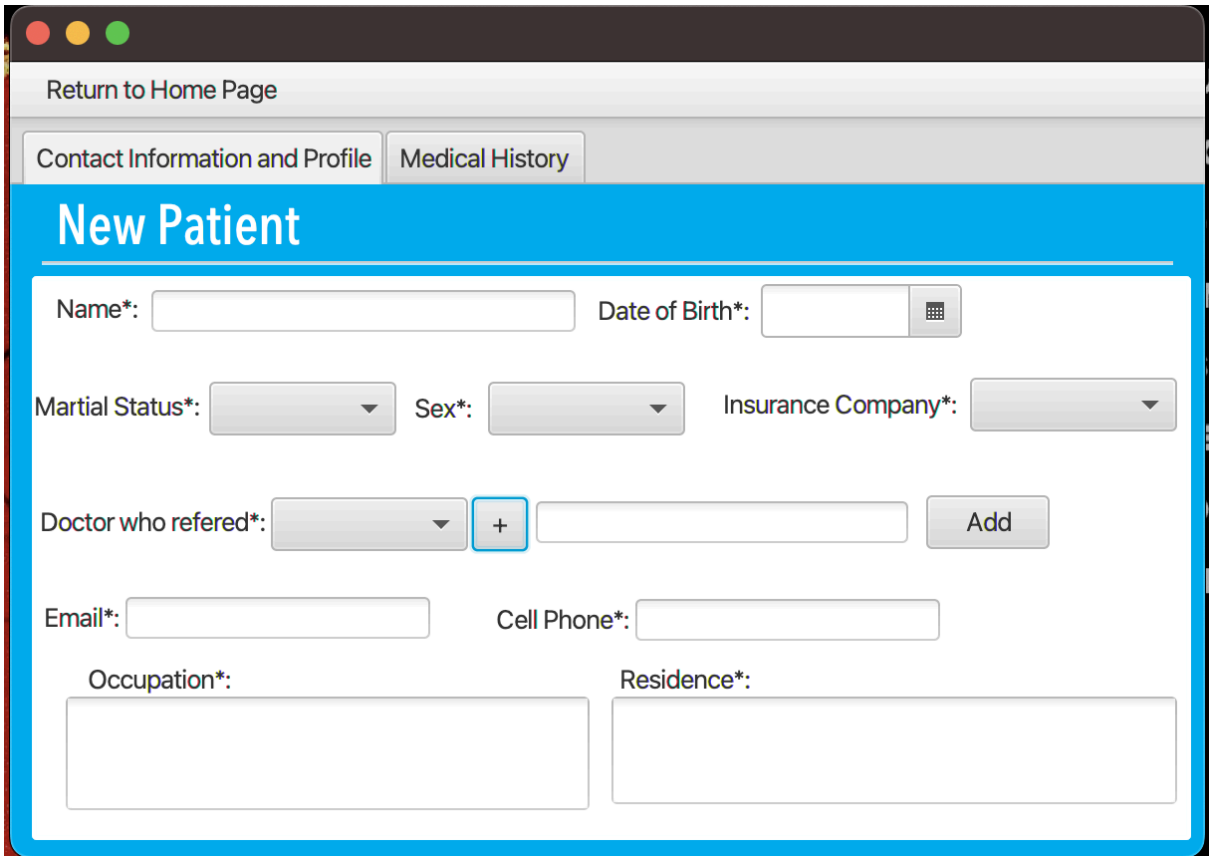
            // Updating CSV file and tables across the program.
            application.Main.updatePatientsAlCsvFile();
            updateNewPatientTable(application.Main.patientsAl.getPatientAl());
            application.Main.nC.updateNewConsultationTable(application.Main.patientsAl.getPatientAl());

            clearTemplate();
            tabPane.getSelectionModel().select(tabTablePatients);
        } else {
            System.out.println("no se edito ;(");
        }
    }
    TableViewSelectionModel<Patient> selectionModel = tablePatients.getSelectionModel();
    selectionModel.clearSelection();

}
```



I decided to not allow the user to change the name of an already registered Patient because it would facilitate the task of looking for a patient using a binary search by name.



The user is able to create a new referring doctor in the New Patient Scene as well as in the View Patients Scene in an attempt to increase efficiency when using this app.

```
void btnPlusReferringDocOnClicked(ActionEvent event) {
    setPlusDoctor(true);
    if (plusActivated == true) {
        setPlusDoctor(true);
        plusActivated = false;
    } else if (plusActivated == false) {
        setPlusDoctor(false);
        plusActivated = true;
    }

}
```

Since this is not a mandatory field to fill, this section can be displayed and hidden whenever the user decides to. This is to ensure a better UX experience by having a cleaner scene.

```
void btnDeleteOnClick(ActionEvent event) {

    Patient temp = tablePatients.getSelectionModel().getSelectedItem();

    if (temp != null) {
        Alert deletionConfirmation = new Alert (AlertType.CONFIRMATION);
        deletionConfirmation.setTitle("CONFIRMATION");
        deletionConfirmation.setHeaderText(null);
        deletionConfirmation.setContentText("Are you sure yo want to delete this patient?");
        deletionConfirmation.showAndWait();
        if (deletionConfirmation.getResult() == ButtonType.OK) {
            application.Main.patientsAl.getPatientAl().remove(temp);
            application.Main.updatePatientsAlCsvFile();
            updateNewPatientTable(application.Main.patientsAl.getPatientAl());
            clearTemplate();
            tabPane.getSelectionModel().select(tabTablePatients);

        }
    } else {
        Alert noPatientFoundAlert = new Alert(AlertType.INFORMATION);
        noPatientFoundAlert.setTitle("No Patient Found");
        noPatientFoundAlert.setHeaderText(null);
        noPatientFoundAlert.setContentText("Please select a patient");
        noPatientFoundAlert.showAndWait();
    }
}
```

I considered it crucial to include a Confirmation alert any time the user is about to delete something because this is a way to ensure that the user is sure about their decision and decrease the possibility of deleting important information accidentally or unintentionally.

```
void btnSearch(ActionEvent event) {
    int index = application.Main.consultationsAl.LinearSearchByNameAndDate(txtSearchName.getText(),dpDateSearch.getValue());
    TableViewSelectionModel<Consultation> selectionModel = tableConsultations.getSelectionModel();
    if (index == -1) {
        Alert patientNotFound = new Alert(AlertType.ERROR);
        patientNotFound.setTitle("No Patient Found");
        patientNotFound.setHeaderText(null);
        patientNotFound.setContentText("This patient was not found, please try again or schedule another consultation.");
        patientNotFound.showAndWait();
    }else {
        selectionModel.select(index);
        System.out.println(application.Main.consultationsAl.getConsultation(index));
    }
    txtSearchName.setText(null);
}
```

To look for a specific consultation in the View Consultations scene, the user must input two pieces of information for the search to start. I decided to consider two parameters in the search process since I considered there is a great possibility of two or more consultations with the same patient existing. Adding the date parameter makes the search more specific for the user to find the exact consultation the user wishes to take a look at, edit, or delete.

Algorithms.

Search

```
void btnSearchClicked(ActionEvent event) {
    int index = application.Main.patientsAl.binarySearchByName(txtSearchPatient.getText());
    TableViewSelectionModel<Patient> selectionModel = tablePatients.getSelectionModel();
    if(index == -1) {
        Alert patientNotFound = new Alert(AlertType.ERROR);
        patientNotFound.setTitle("No Patient Found");
        patientNotFound.setHeaderText(null);
        patientNotFound.setContentText("This patient was not found, please try again or register a new patient.");
        patientNotFound.showAndWait();
    }else {
        selectionModel.select(index);
    }

}
```

```
    public int binarySearchByName(String name) {
        selectionSortByName();
        int start = 0;
        int end = PatientAl.size()-1;
        int mid;

        while (start <= end ) {
            mid = (start + end) /2;
            if (name.compareTo(PatientAl.get(mid).getName()) == 0) {
                return mid;
            }
            if (name.compareTo(PatientAl.get(mid).getName()) < 0) {
                end = mid - 1;
            }else {
                start = mid + 1;
            }
        }
        return -1;
    }
}
```

Binary Search was used several times throughout the code. I was informed by my client that he has treated privately more than 2000 patients throughout his career. Since this ArrayList could potentially include a large amount of elements, I decided to use a binary search algorithm in an attempt to save RAM, and computing power and consequently reduce energy consumption. Also, the complexity of this searching algorithm is O(log n), which increases efficiency when looking for an element, finding the desired element in a very small amount of time.

```
void btnSearch(ActionEvent event) {
    int index = application.Main.consultationsAl.LinearSearchByNameAndDate(txtSearchName.getText(),dpDateSearch.getValue());
    TableViewSelectionModel<Consultation> selectionModel = tableConsultations.getSelectionModel();
    if (index == -1) {
        Alert patientNotFound = new Alert(AlertType.ERROR);
        patientNotFound.setTitle("No Patient Found");
        patientNotFound.setHeaderText(null);
        patientNotFound.setContentText("This patient was not found, please try again or schedule another consultation.");
        patientNotFound.showAndWait();
    }else {
        selectionModel.select(index);
        System.out.println(application.Main.consultationsAl.getConsultation(index));
    }
    txtSearchName.setText(null);
}
```

```
public int LinearSearchByNameAndDate(String name, LocalDate date) {
    String dateString = date + "";
    selectionSortByDate();
    selectionSortByTime();

for (int i = 0; i < consultationsAl.size(); i++) {
    if (name.equals(consultationsAl.get(i).getName()) && dateString.equals(consultationsAl.get(i).getConsultationDate())) {
    return i;
    }
}
    return -1;
}
```

In the View Consultations Scene, the search method utilizes linear search to find the desired consultation by the user. This is because this search uses two parameters to look for, which is easier to implement in a linear search algorithm (This decision was explained in the previous section). Since the user is encouraged to constantly delete past consultations from the View Consultations TableView, the Array List is expected to constantly remain short. This fact makes it unnecessary to consider aspects such as memory usage, energy, and search efficiency, because the complexity of the Array List will remain low, and would not make a great difference if I were to use more efficient algorithms.

Selection Sort

```
public void selectionSortByDate() {
    for (int i = 0; i < consultationsAl.size(); i++) {
        for (int j = i+1; j < consultationsAl.size(); j++) {
            if (consultationsAl.get(i).getConsultationDate().compareToIgnoreCase(consultationsAl.get(j).getConsultationDate()) > 0) {
                Consultation swapper = consultationsAl.get(i);
                consultationsAl.set(i, consultationsAl.get(j));
                consultationsAl.set(j, swapper);
            }
        }
    }
}
public void selectionSortByTime() {
    for (int i = 0; i < consultationsAl.size(); i++) {
        for (int j = i+1; j < consultationsAl.size(); j++) {
            if (consultationsAl.get(i).getConsultationTime().compareToIgnoreCase(consultationsAl.get(j).getConsultationTime()) > 0) {
                Consultation swapper = consultationsAl.get(i);
                consultationsAl.set(i, consultationsAl.get(j));
                consultationsAl.set(j, swapper);
            }
        }
    }
}
```

Selection sort was used several times throughout the code, thanks to its easy implementation and the great complexity of $O(n^2)$. Sorting had two main purposes. The first one was to sort information on all TableViews included all around the application to make it easier for the user to look for an element manually. This was especially important in the View Consultations TableView, where the user can see the consultations ordered by date, having the nearest consultation on top. Its second purpose was to sort elements in the array list to be able to use binary search.

Tools

For this application, I used GUI using JavaFX. I considered that Java was the most ideal programming language to use thanks to its OOP characteristics. I used specifically JavaFX8 because this version includes useful features such as the date picker, which was used several times throughout the code. Using GUI with Scene Builder facilitated the design process, and saved time when developing the aesthetics part.

```
import java.time.LocalDate;
import java.time.LocalTime;
import java.time.format.DateTimeFormatter;
import java.util.ArrayList;
```

```
public int getAge() {
    LocalDate currentDate = LocalDate.now();
    Period period = Period.between(dateOfbirth, currentDate);
    age = period.getYears();
    return age;
}
```

```
public void updateEndingTime() {
    DateTimeFormatter formatter = DateTimeFormatter.ofPattern("HH:mm");
    LocalTime time = LocalTime.parse(consultationTime, formatter);
    consultationEndingTime = time.plusHours(2) + "";
}
```

I used the Java package time which was introduced in Java 8, which helped with some instance variables, such as the date of birth in the patient class, and the date and time of consultations in the Consultations class. This facilitated processes, and made information more precise and accurate, for example, when calculating someone's age considering real-time.

Word Count: 1106